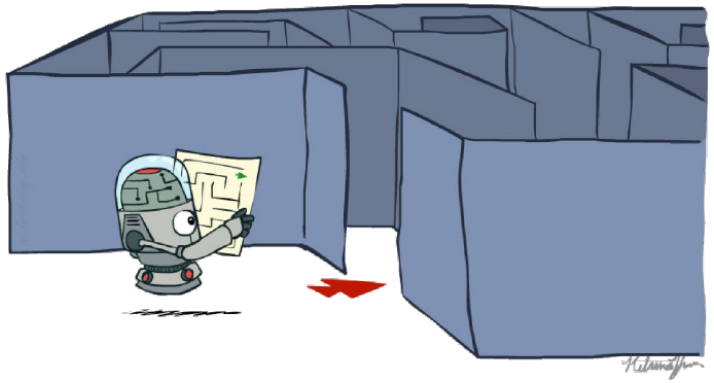
Artificial Intelligence
Chapter 2 - Search

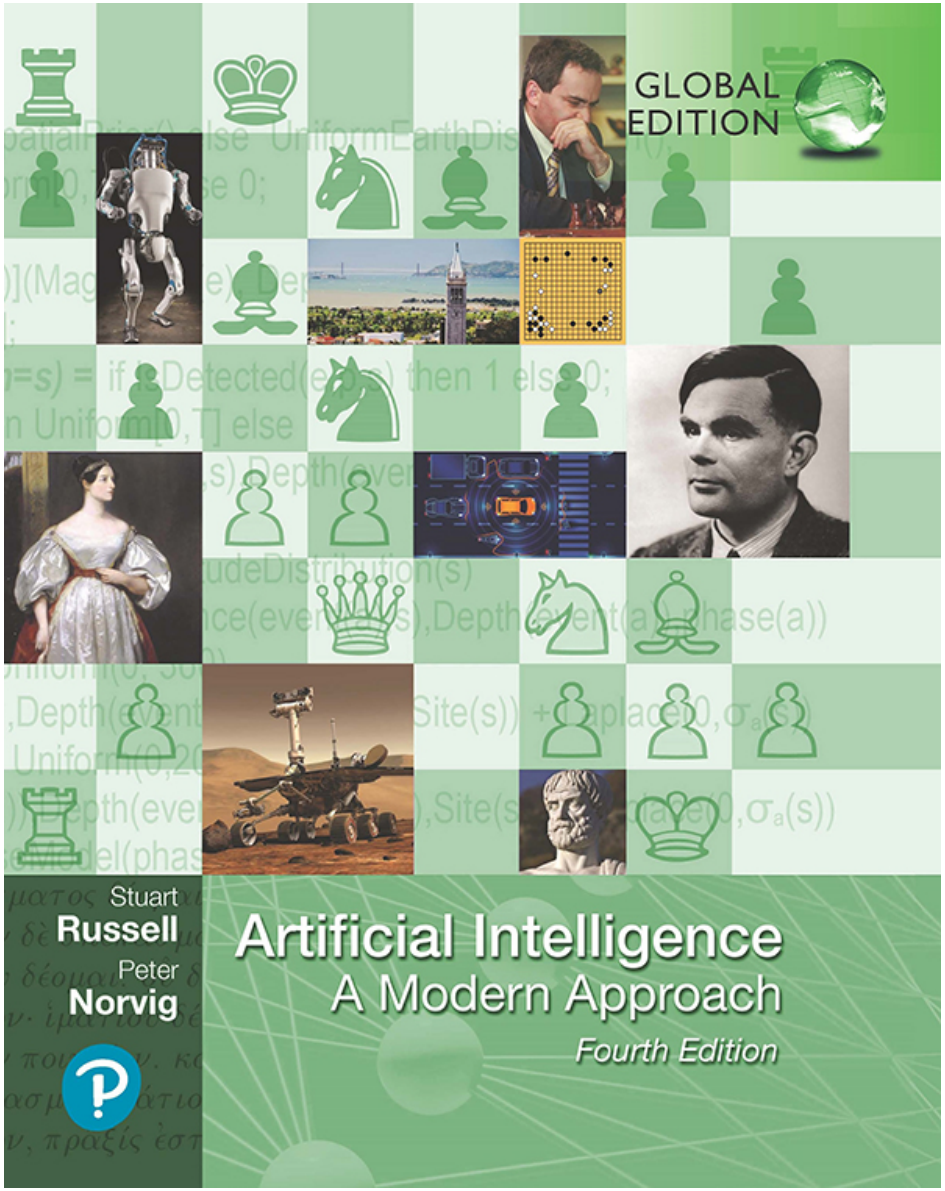
Jean-Sébastien Lerat



HEH be
Sciences
et technologies

2023-2024

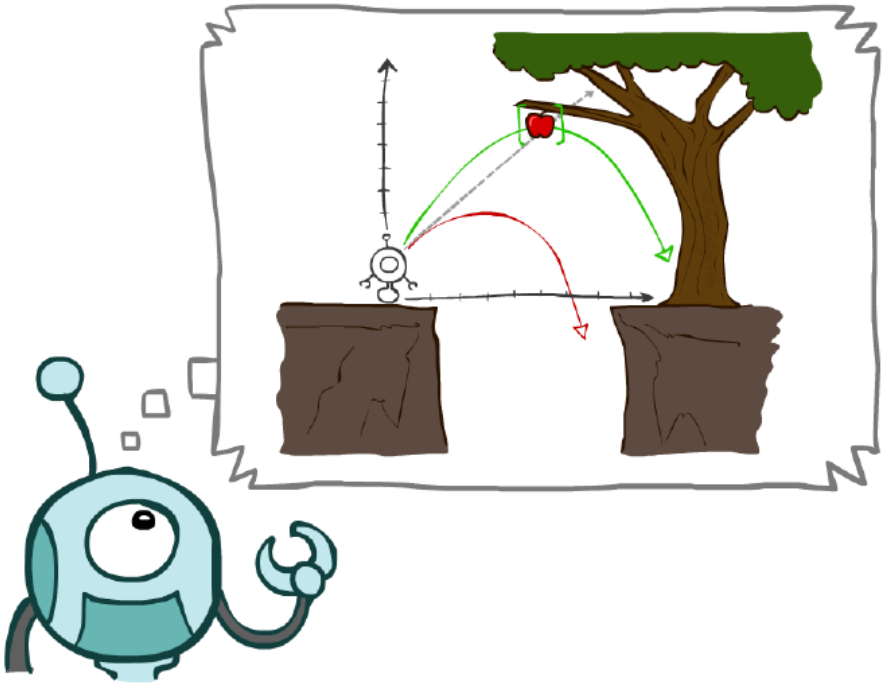
Main Reference



Slides are based on CS 188 of Berkley

Introduction

- Agents that Plan Ahead
- Search Problems
- Uninformed Search Methods
  - Depth-First Search
  - Breadth-First Search
  - Uniform-Cost Search

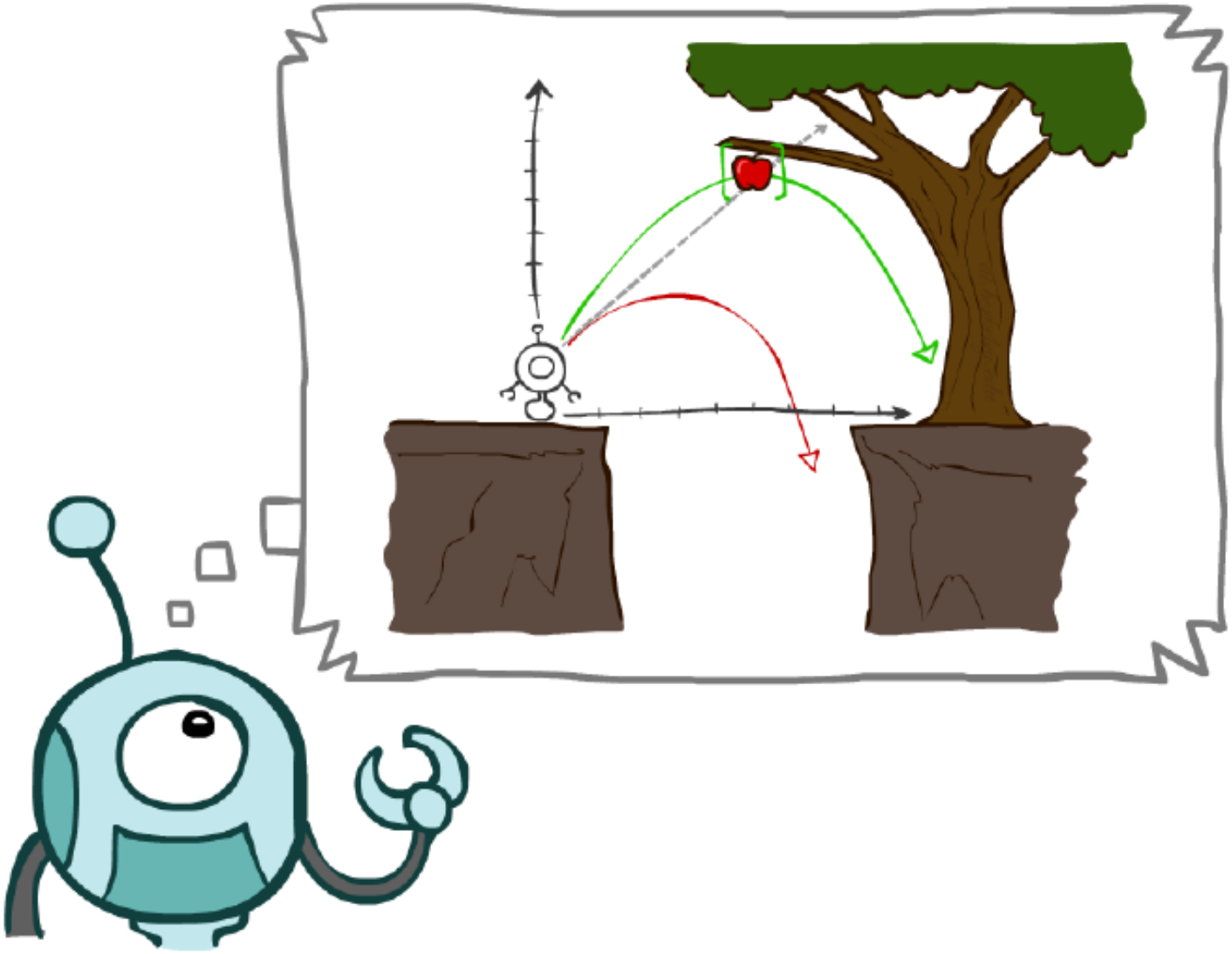


Content

1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

Agent

Planning

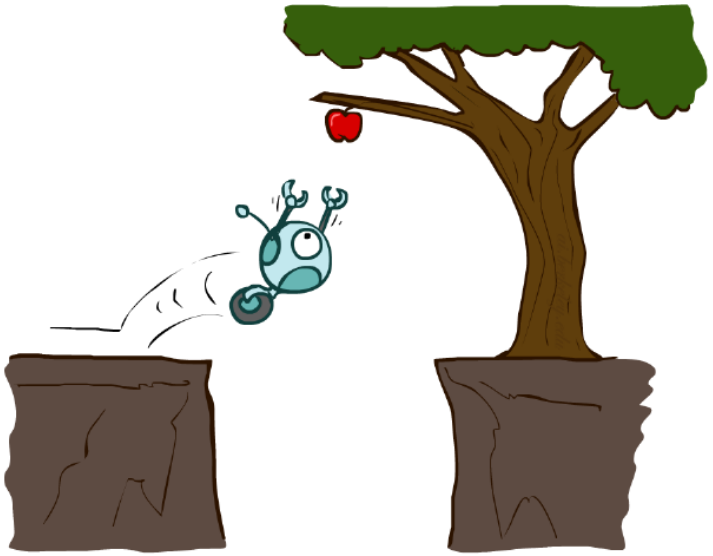


Agent

Reflex Agent

Reflex Agents

- Choose action based on current percept (and maybe memory)
- May have memory or a model of the world's current state
- Do not consider the future consequences of their actions
- Consider how the world IS



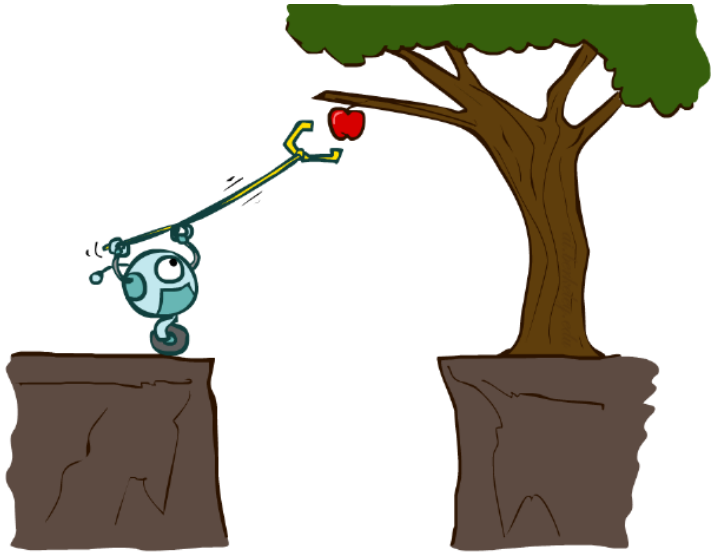
Can a reflex agent be rational?

Agent

Planning Agent

Planning Agents

- Ask "what if"
- Decisions based on (hypothesized) consequences of actions
- Must have a model of how the world evolves in response to actions
- Must formulate a goal (test)
- Consider how the world WOULD BE



Optimal vs. complete planning

Planning vs. replanning

Content

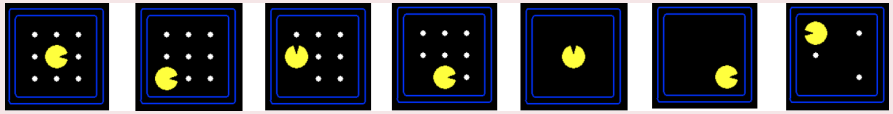
1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

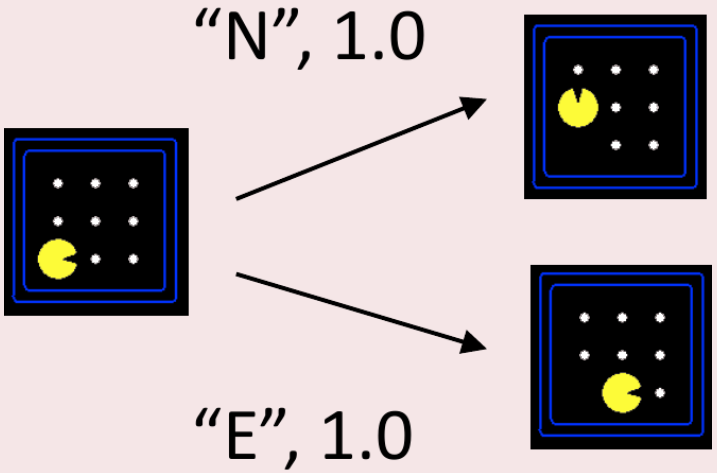
Search Problem

Search Problem

A **search problem** consists of

A state space





A successor function (with actions, costs)

A start state and a goal test

A **solution** is

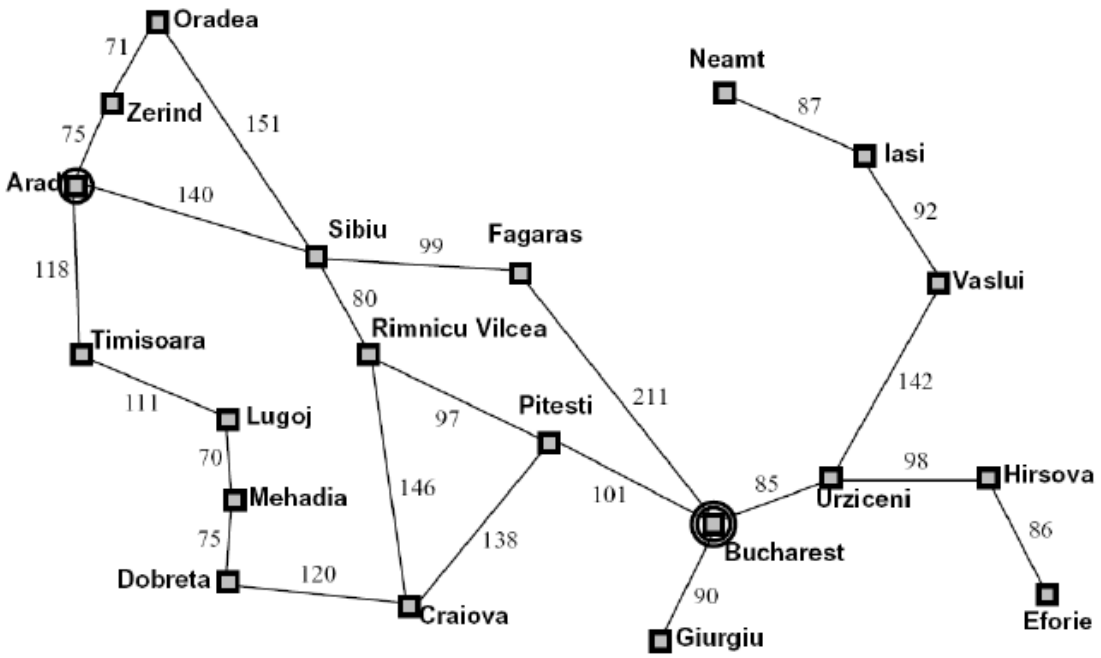
a sequence of actions (a plan) which transforms the start state to a goal state

Search Problem

Search Problems Are Models

Search Problem

Example: Traveling in Romania



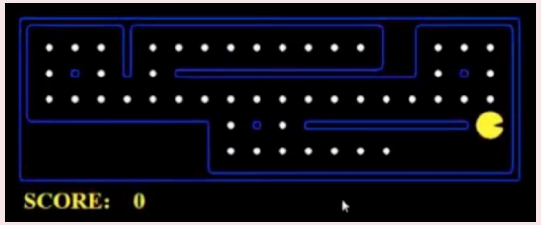
- **State space** cities
- **Start state** Arad
- **Goal test**
  Is state == Bucharest?
- **Successor function**
  Roads: Go to adjacent city with
  cost = distance
- **Solution** ?

Search Problem

State Space

The **world state**

includes every last detail of the environment



A **search state**

keeps only the details needed for planning (abstraction).

Problem: Pathing

- States: (x, y) location
- Actions: North, South, East, West
- Successor: update location only
- Goal test: is (x, y)=END

Problem: Eat-All-Dots

- States: (x,y), dot booleans
- Actions: NSEW
- Successor: update location and possibly a dot boolean
- Goal test: dots all false

Search Problem

State Size

World state:

Agent positions 120

Food count 30

Ghost positions 12

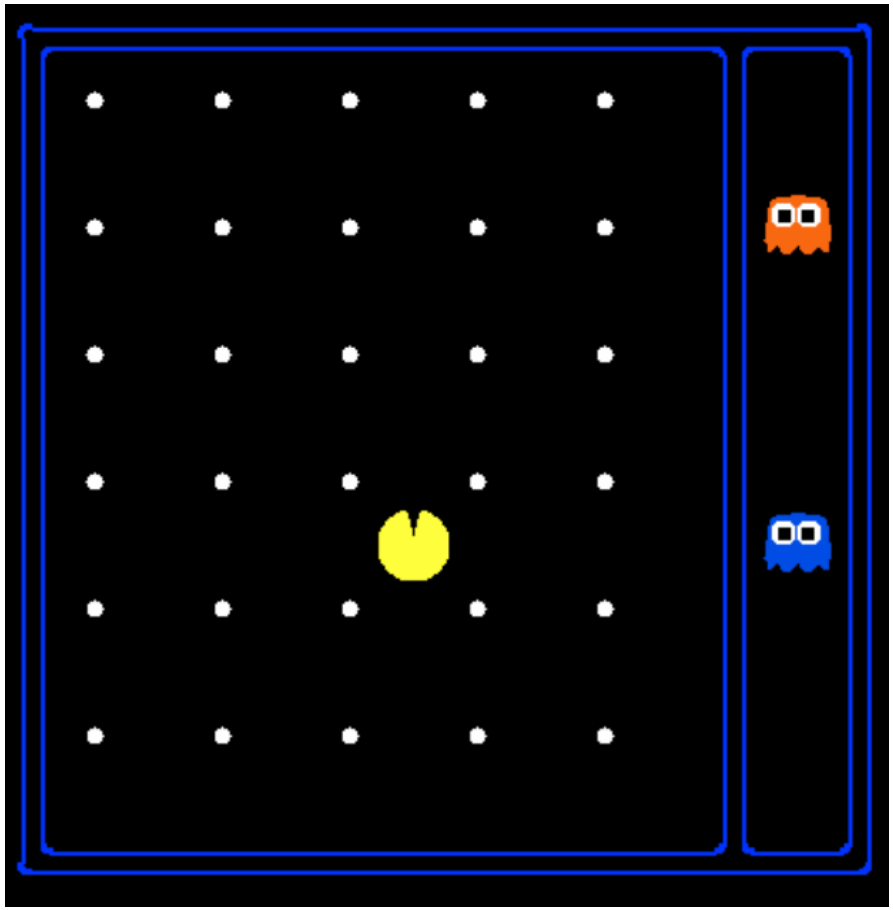
Agent facing NSEW

How many

World states $120 \times \left(2^{30}\right) \times \left(12^{2}\right) \times 4$

States for pathing 120

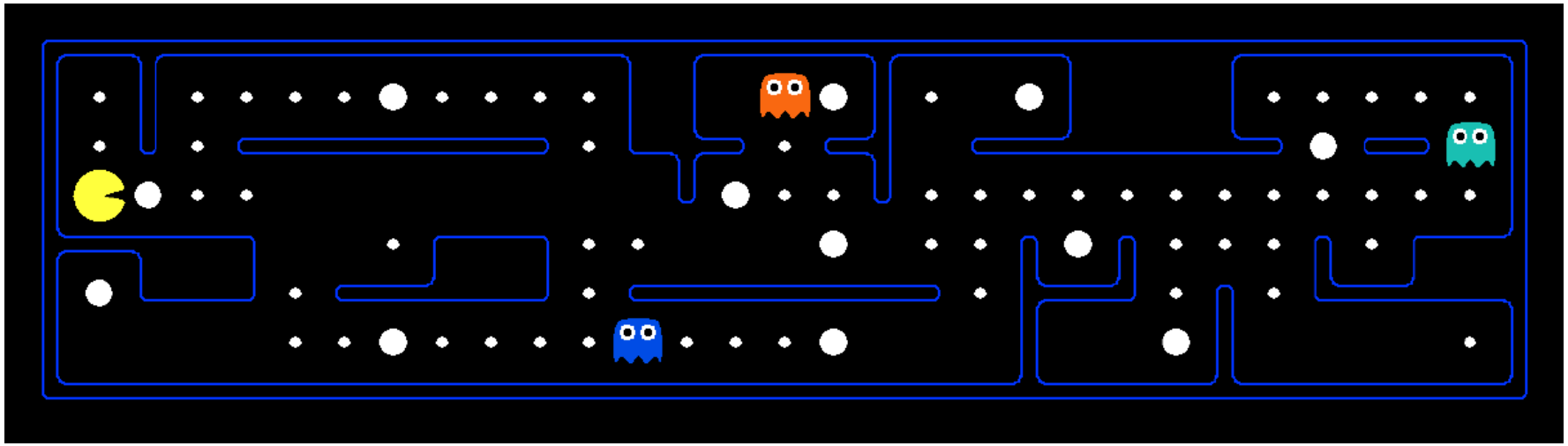
States for eat-all-dots $120 \times \left(2^{30}\right)$



Search Problem

Quiz: Safe Passage

Problem: *eat all dots while keeping the ghosts perma-scared*

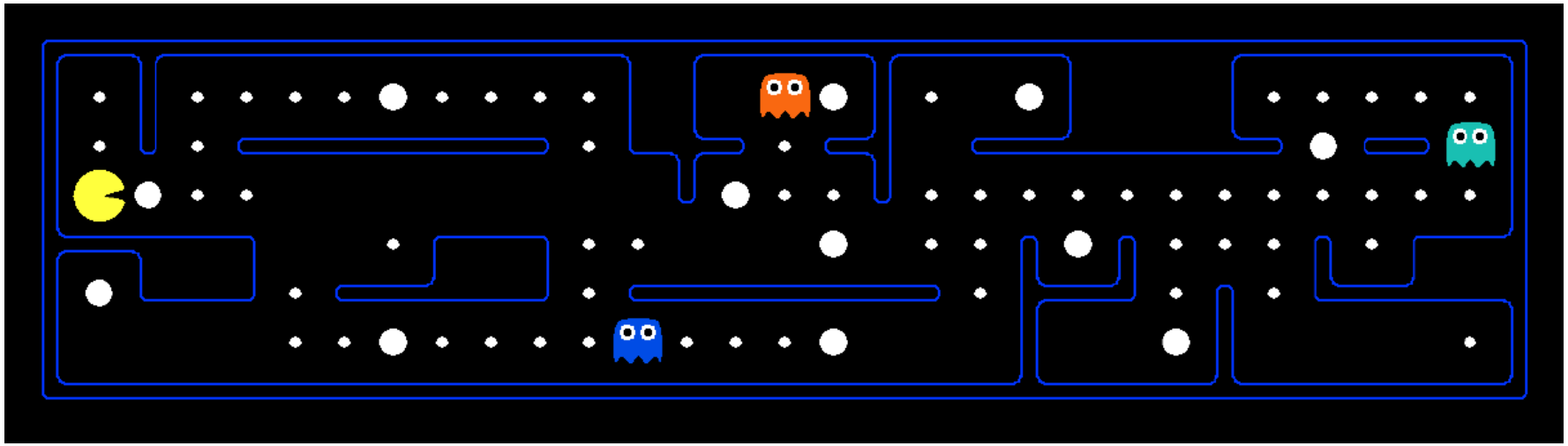


What does the state space have to specify?

Search Problem

Quiz: Safe Passage

Problem: *eat all dots while keeping the ghosts perma-scared*

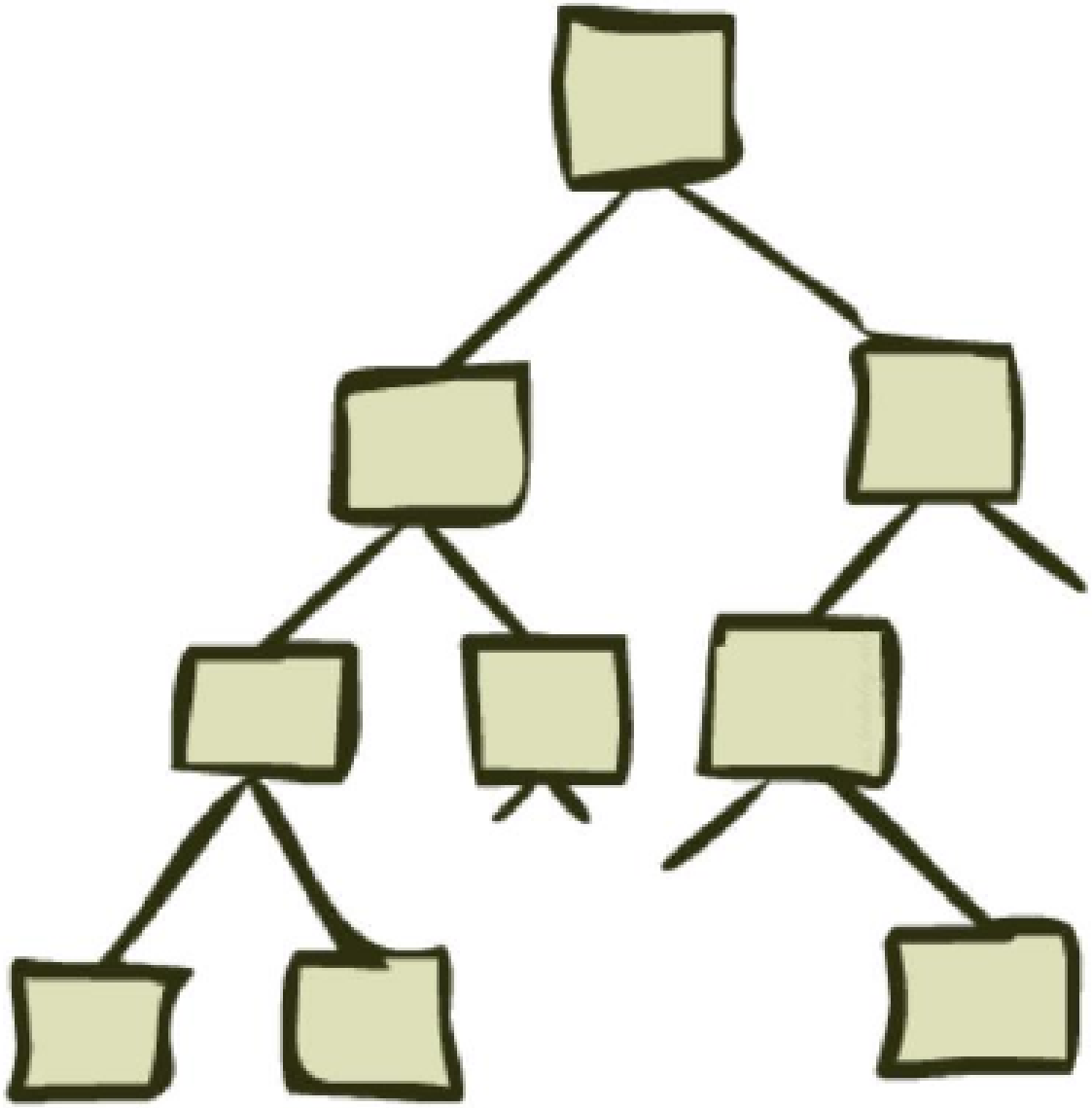


What does the state space have to specify?

(agent position, dot booleans, power pellet booleans, remaining scared time)

Search Problem

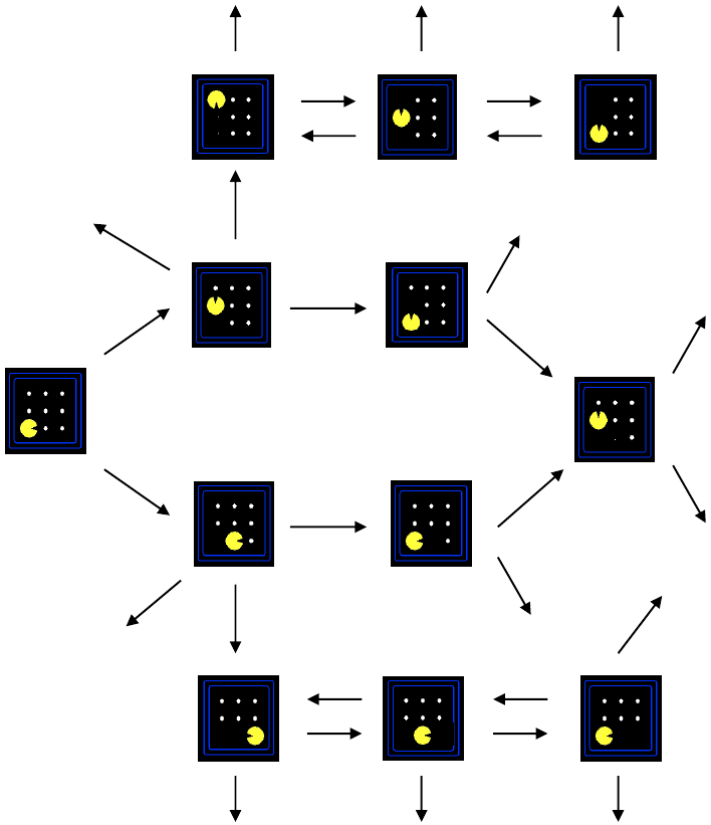
Data Structure



Search Problem

Data Structure: Graph

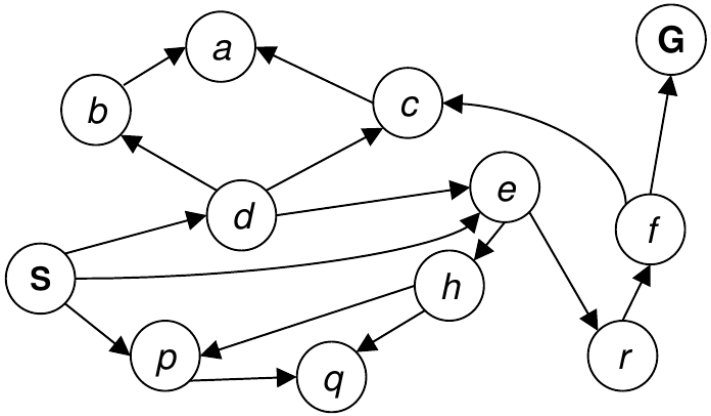
- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- Rarely build this full graph in memory (it's too big), but it's a useful idea



Search Problem

Data Structure: Graph

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- Rarely build this full graph in memory (it's too big), but it's a useful idea



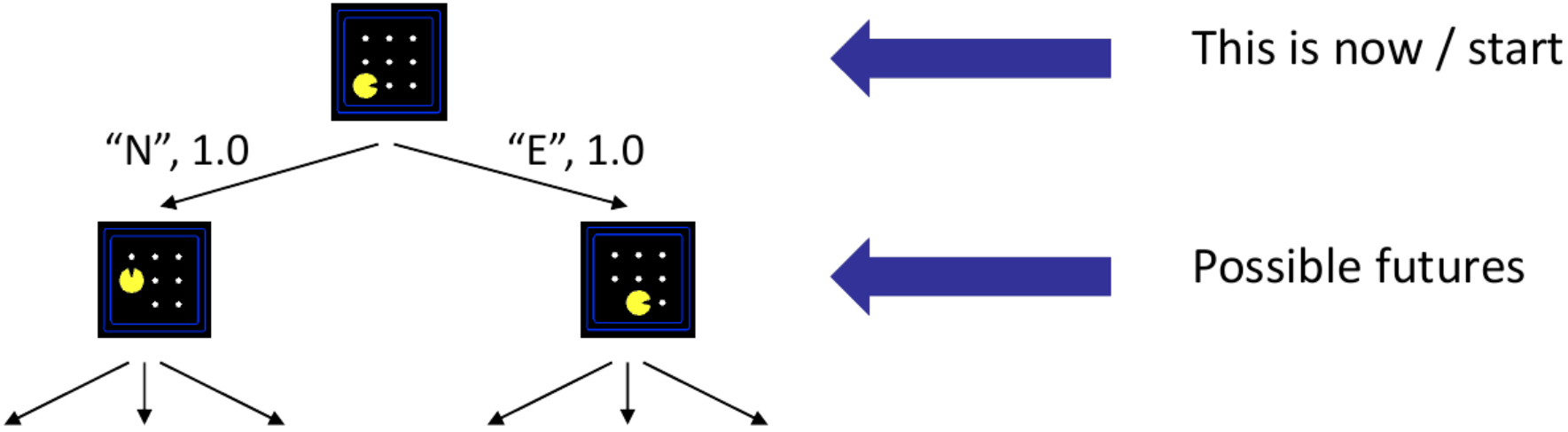
Tiny state space graph for a tiny search problem

Search Problem

Data Structure: Tree

A search tree:

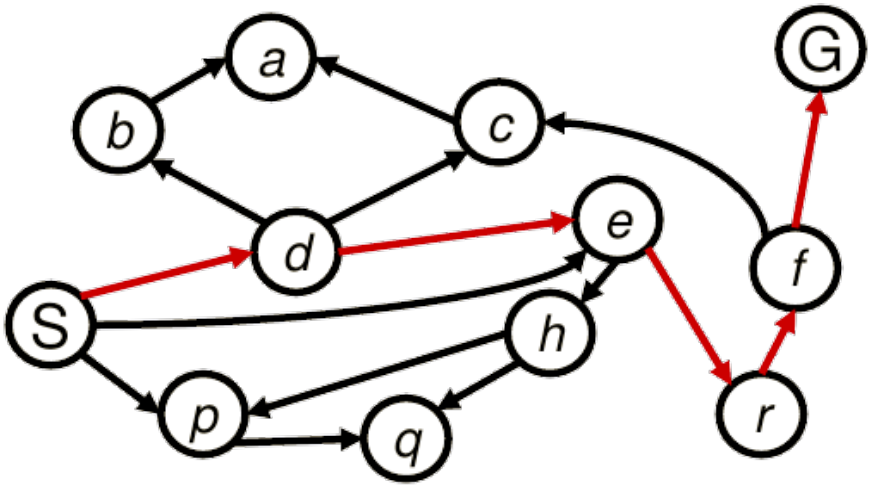
- A "what if" tree of plans and their outcomes
- The start state is the root node
- Children correspond to successors
- Nodes show states, but correspond to PLANS that achieve those states
- For most problems, we can never actually build the whole tree



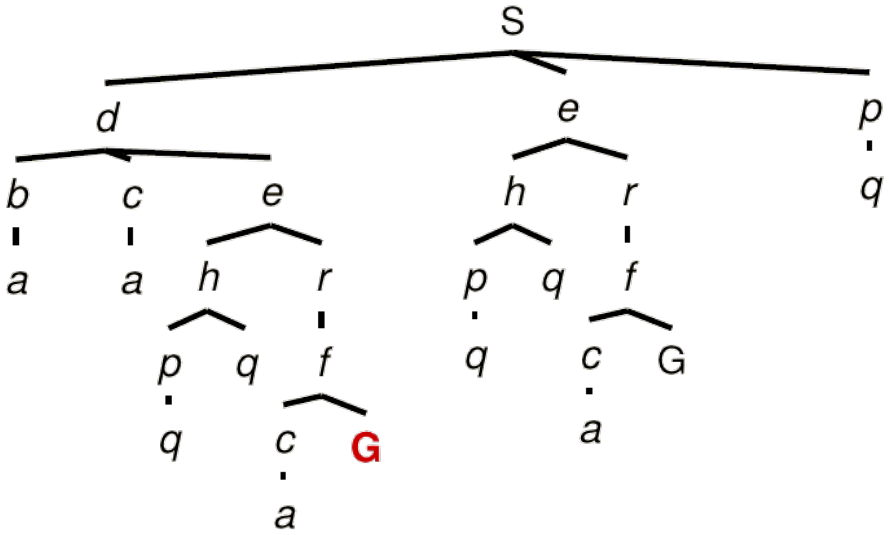
Search Problem

Data Structure: Graph versus Tree

Graph



Tree



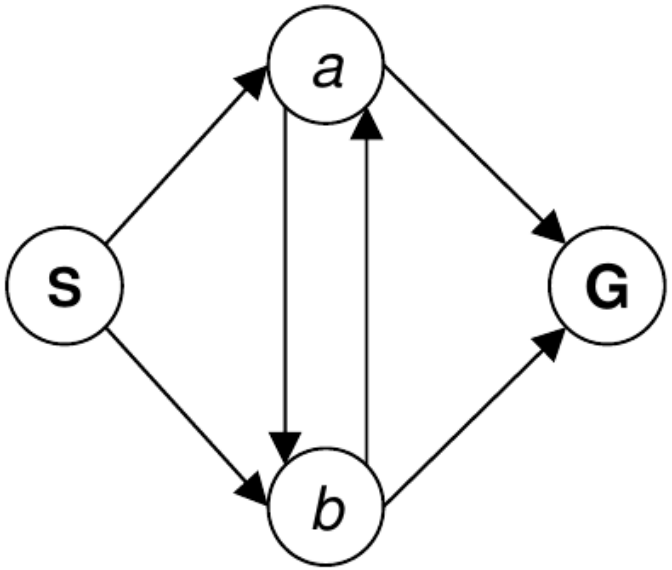
Each node in the search tree is an entire path in the state space graph.

We construct both on demand – and we construct as little as possible.

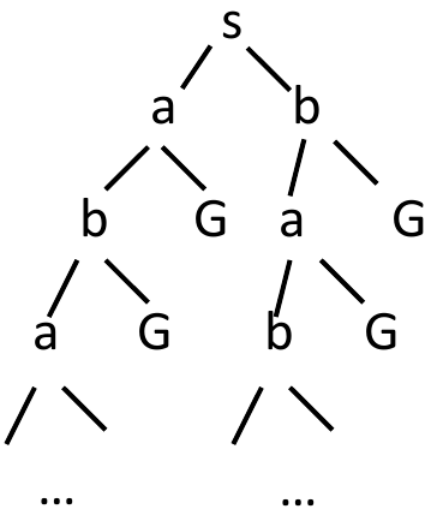
Search Problem

Data Structure: Graph versus Tree

Consider this 4-state graph:



How big is its search tree (from S)?



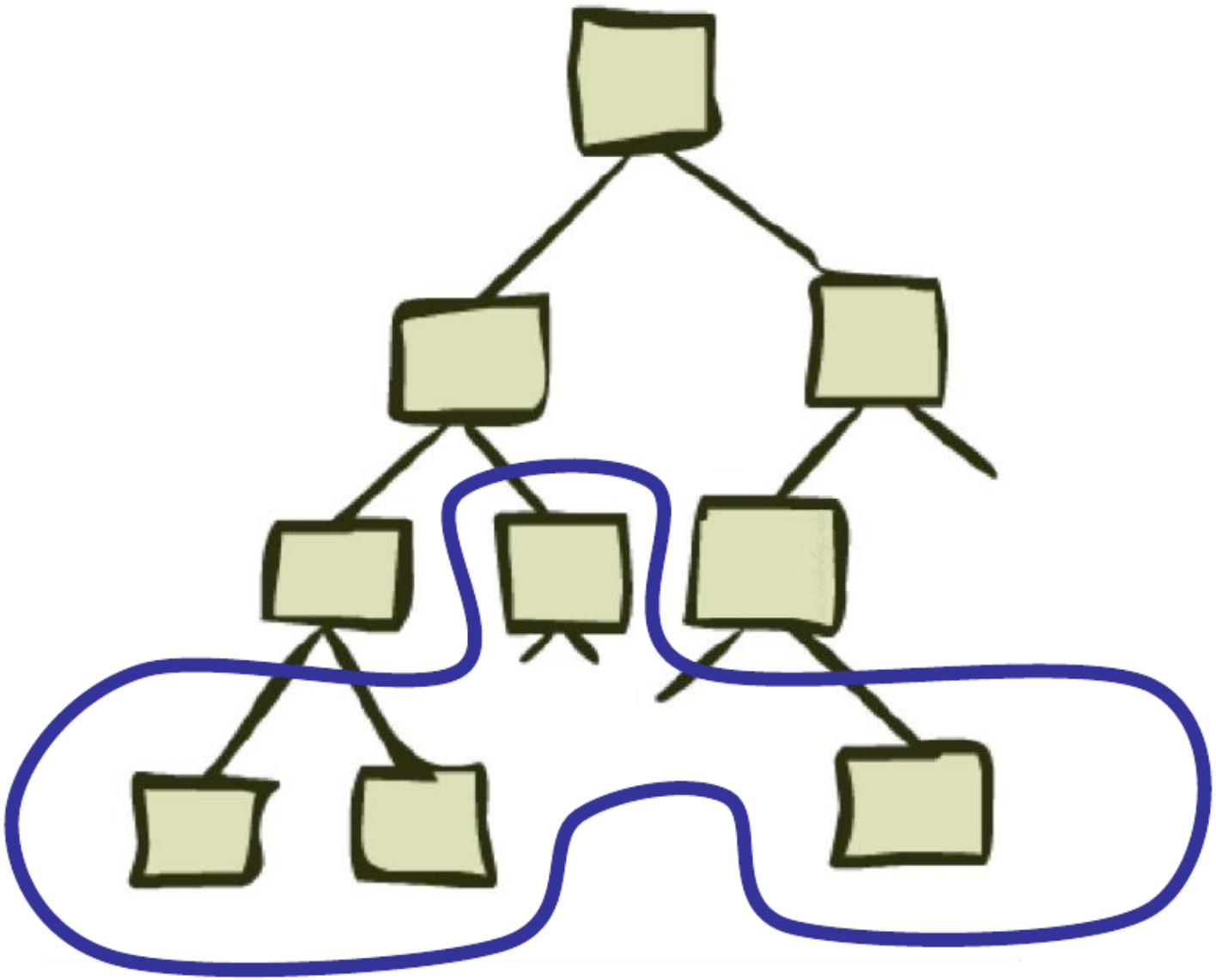
∞

Important: Lots of repeated structure in the search tree!

Content

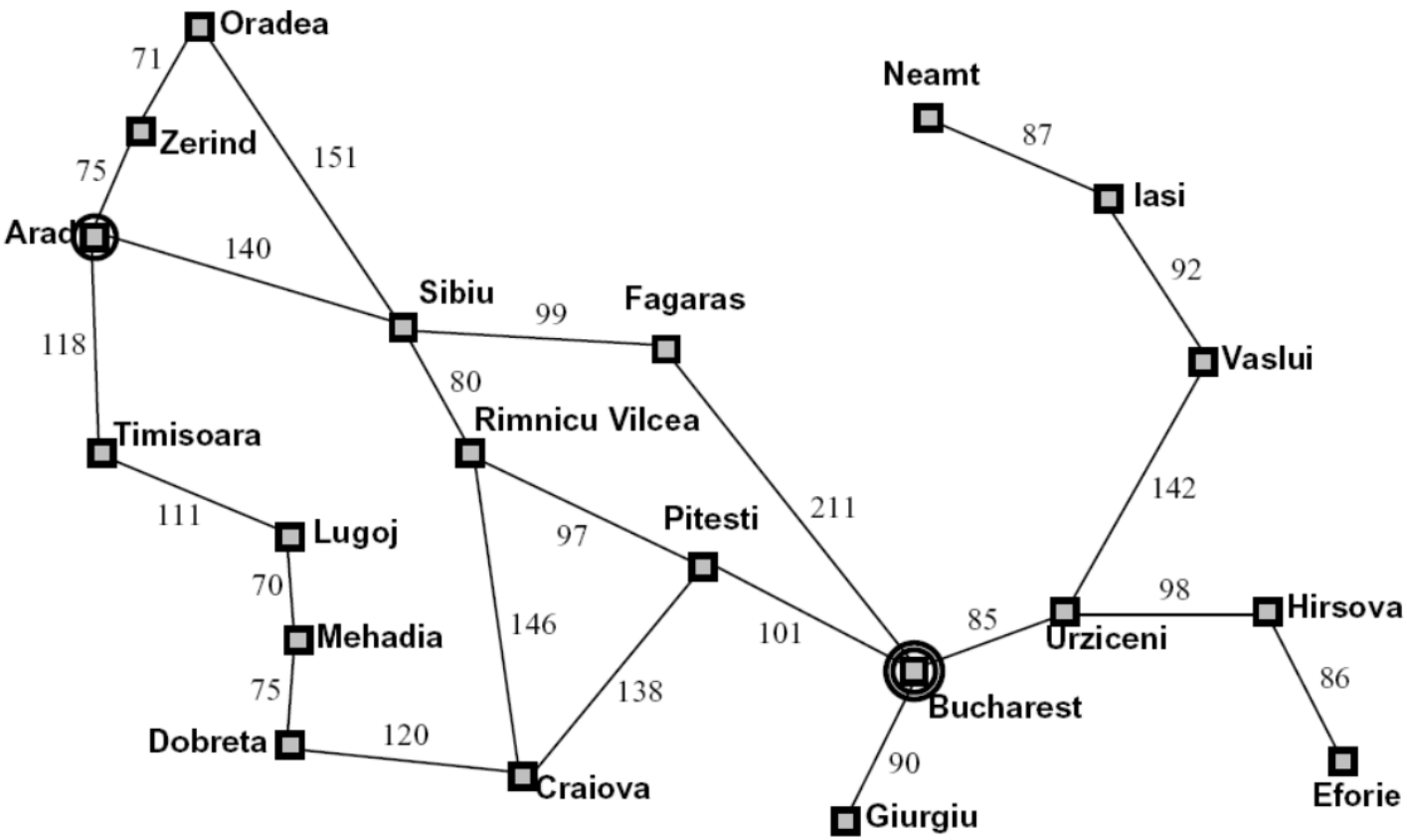
1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

Tree Search



Tree Search

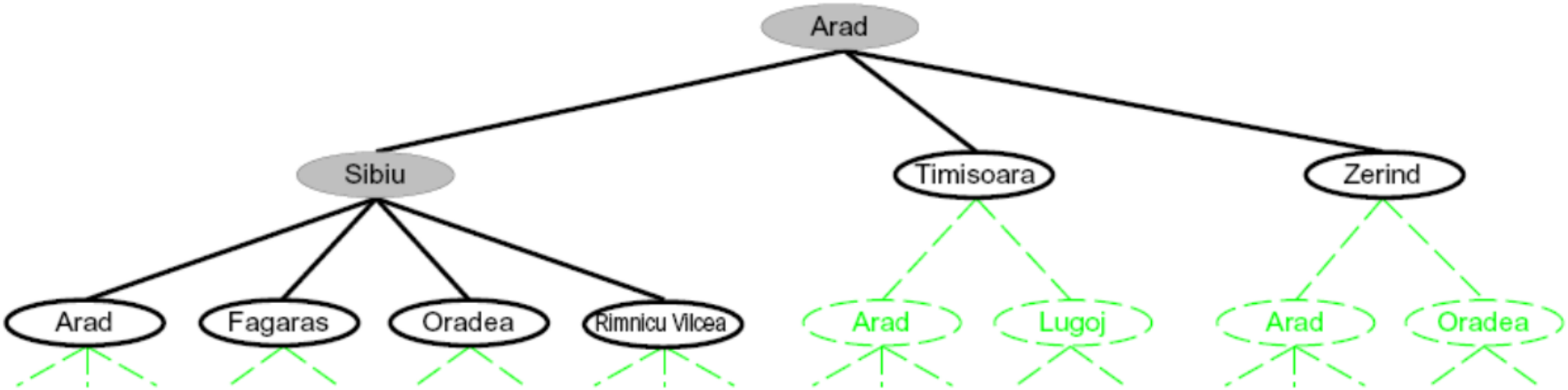
Search Example: Romania



Tree Search

Search:

- Expand out potential plans (tree nodes)
- Maintain a frontier of partial plans under consideration
- Try to expand as few tree nodes as possible



Tree Search

General Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
   initialize the search tree using the initial state of problem
   loop do
       if there are no candidates for expansion then return failure
       choose a leaf node for expansion according to strategy
       if the node contains a goal state then return the corresponding solution
       else expand the node and add the resulting nodes to the search tree
   end
```

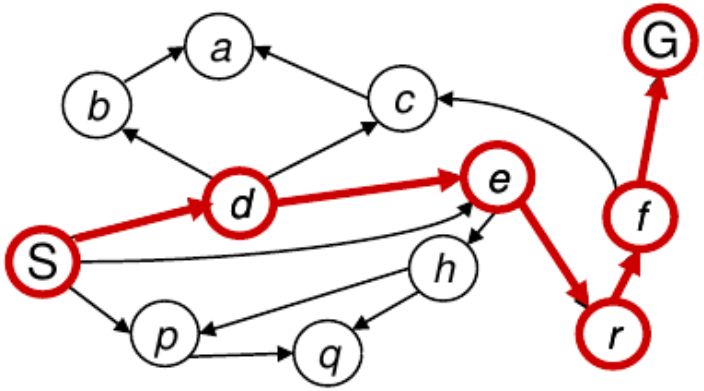
Important ideas:

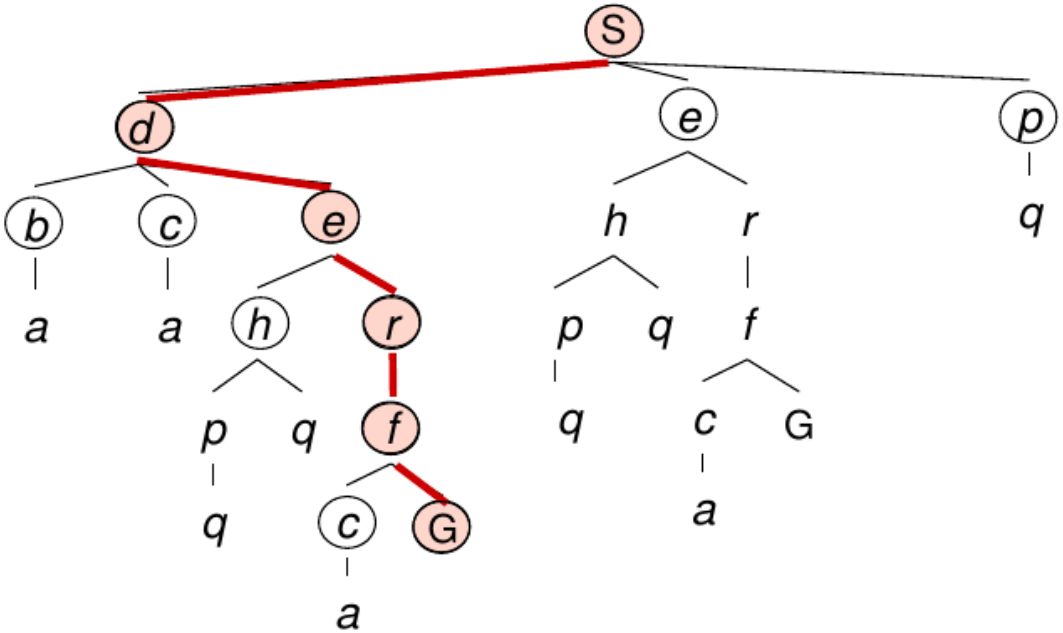
- Frontier
- Expansion
- Exploration strategy

Main question: which frontier nodes to explore?

Tree Search

Example: Tree Search





~~s~~

~~s → d~~

s → e

s → p

s → d → b

s → d → c

~~s → d → e~~

s → d → e → h

~~s → d → e → r~~

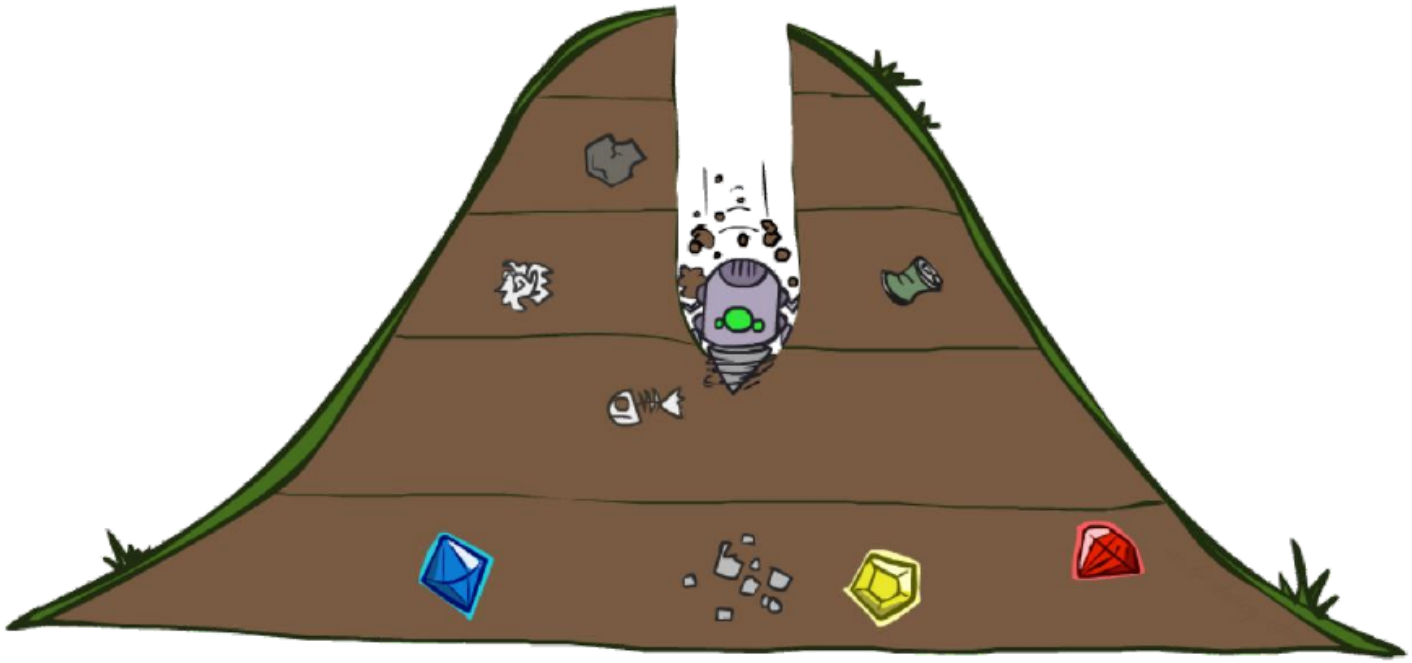
~~s → d → e → r → f~~

s → d → e → r → f → c

~~s → d → e → r → f → G~~

Tree Search

Depth-First Search

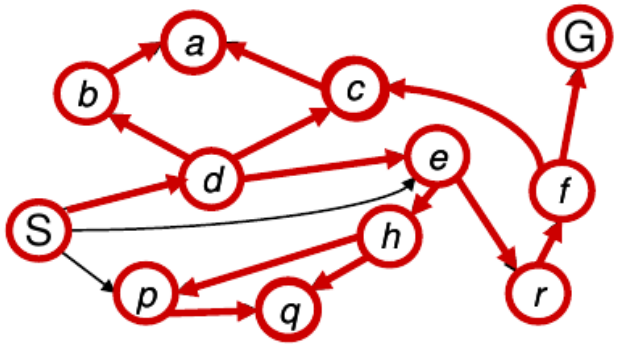


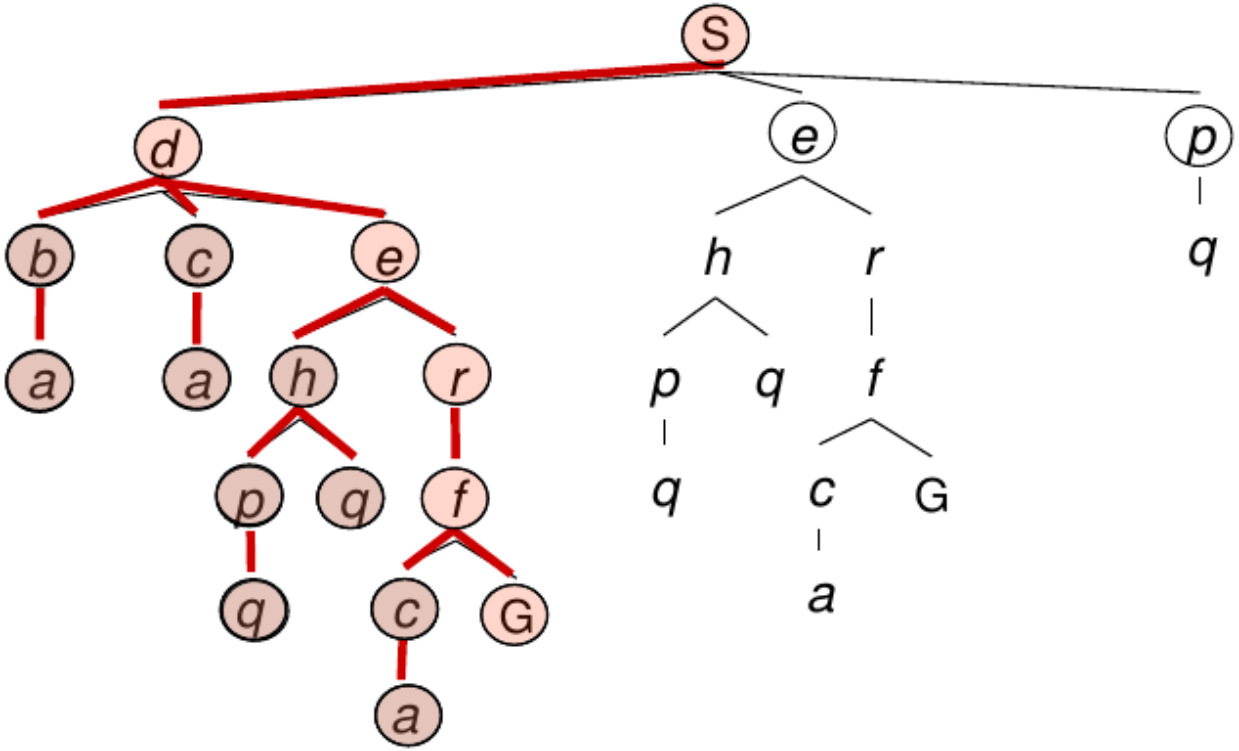
Tree Search

Depth-First Search

Strategy: expand a **deepest** node first

Implementation: Frontier is a **LIFO** stack





Tree Search

Search Algorithm Properties

Complete Guaranteed to find a solution if one exists?

Optimal: Guaranteed to find the least cost path?

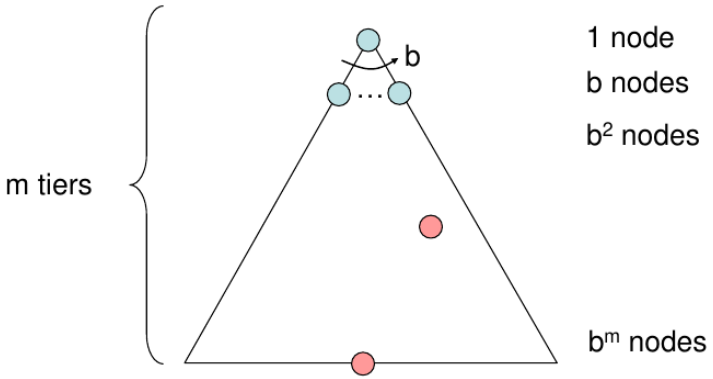
Time complexity

Space complexity

Features
- **b** is the branching factor
- **m** is the maximum depth
- solutions at various depths

Number of nodes (entire tree)

$$1 + b + b^2 \ldots + b^m = O\left(b^m\right)$$



Tree Search

Depth-First Search Properties

What nodes DFS expand?

- Some left prefix of the tree.
- Could process the whole tree!
- If m is finite, takes time $O(b^m)$

How much space does the frontier take?

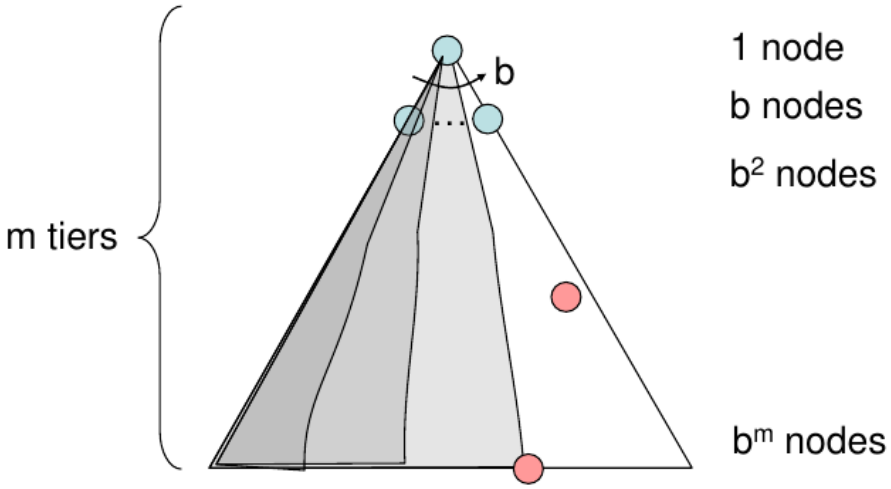
Only has siblings on path to root
$\Longrightarrow O(bm)$

Complete

m could be infinite, so only if we prevent cycles (more later)

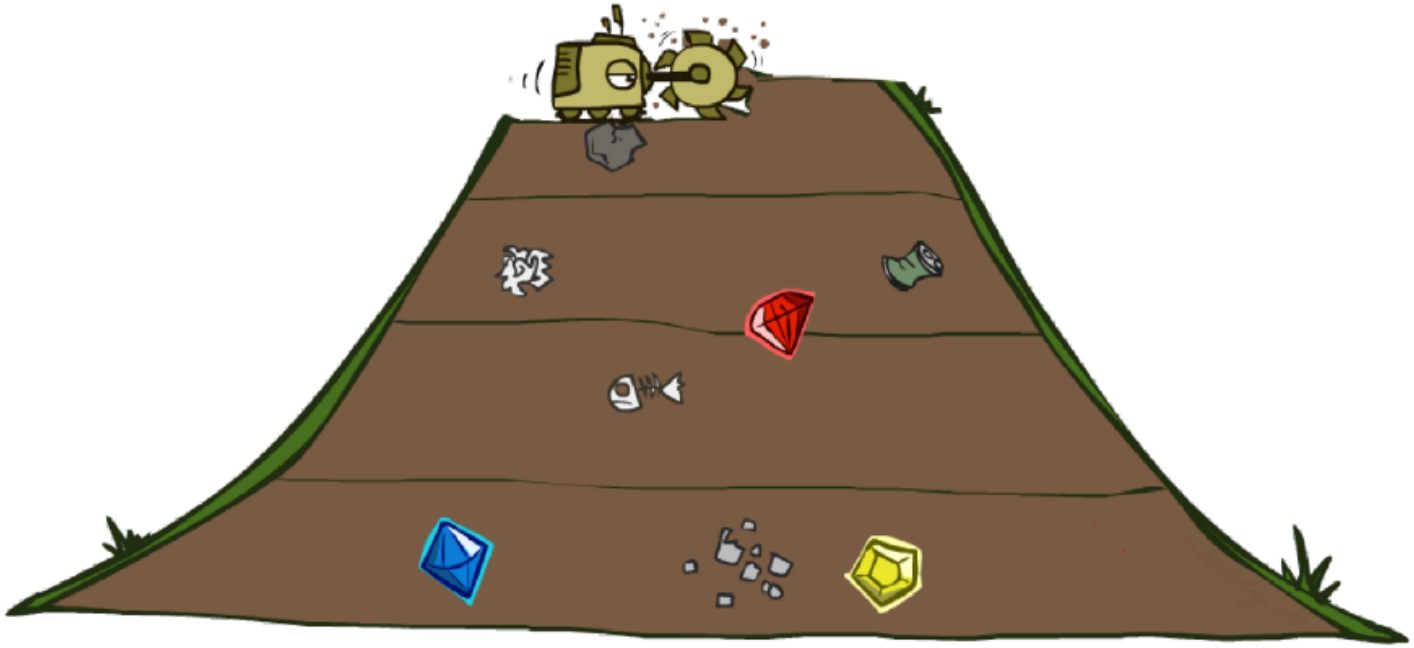
Optimal

No, it finds the *leftmost* solution, regardless of depth or cost



Tree Search

Breadth-First Search

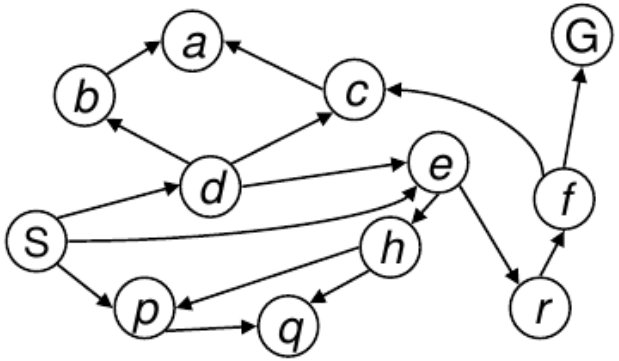


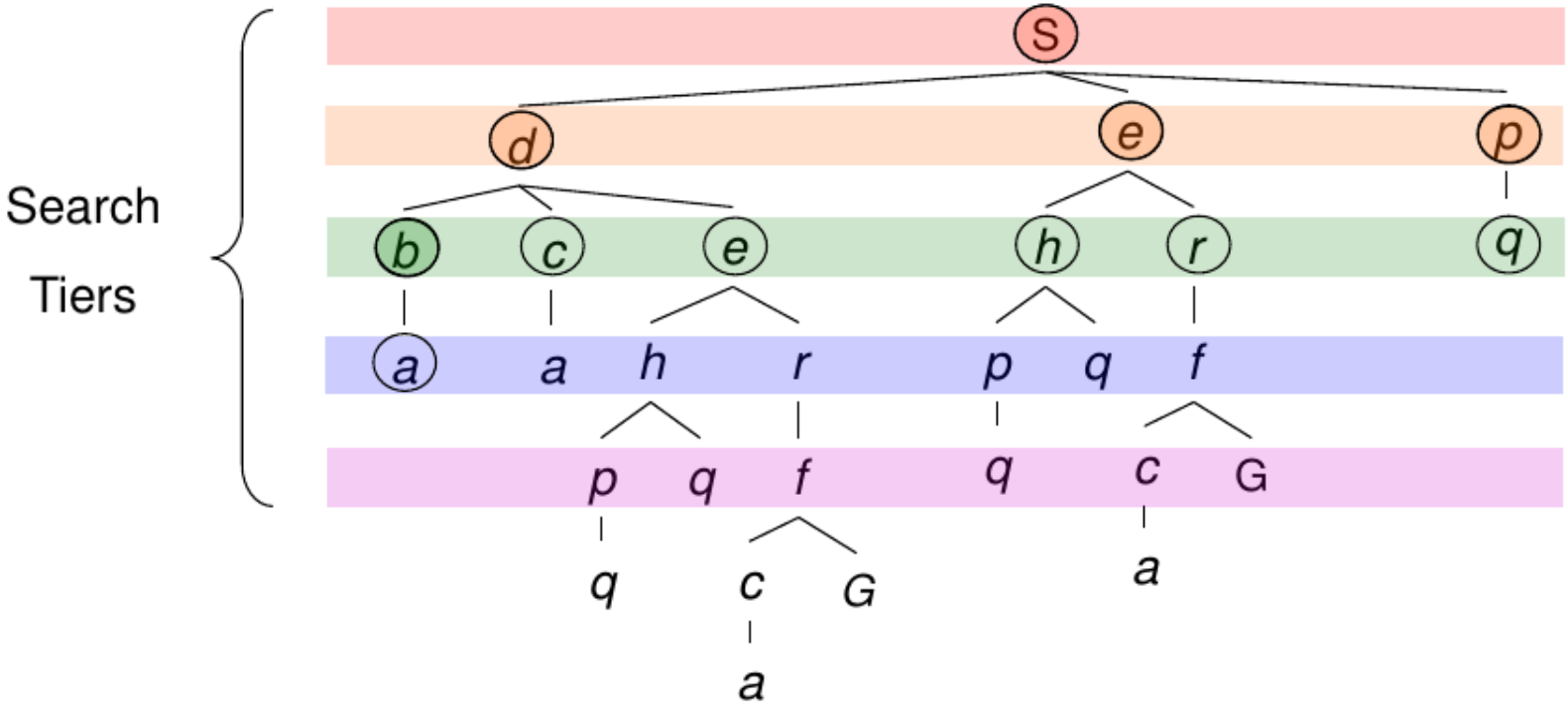
Tree Search

Breadth-First Search

Strategy: expand a **shallowest** node first

Implementation: Frontier is a **FIFO** queue





Tree Search

Breadth-First Search Properties

What nodes BFS expand?

- Processes all nodes above shallowest solution
- Let depth of shallowest solution be s
- Search takes time $O(b^s)$

How much space does the frontier take?

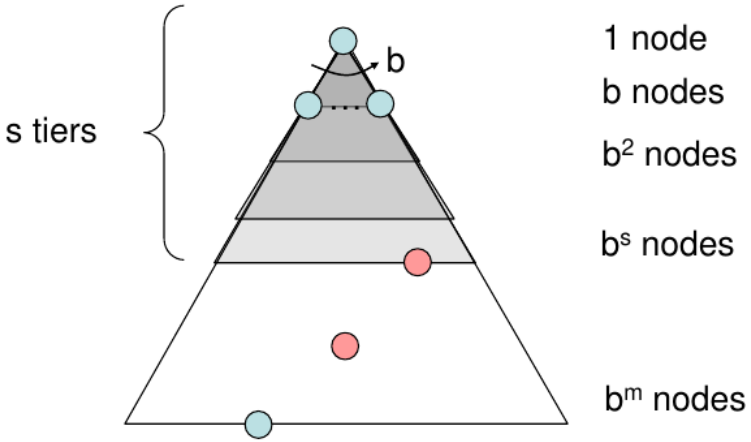
Has roughly the last tier
$\Longrightarrow O(b^s)$

Complete

s must be finite if a solution exists, so yes!

Optimal

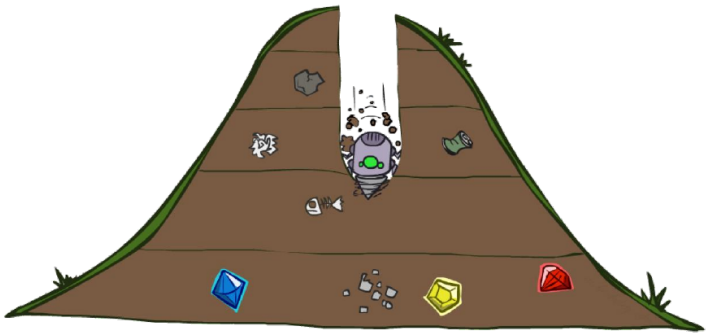
Only if costs are all 1

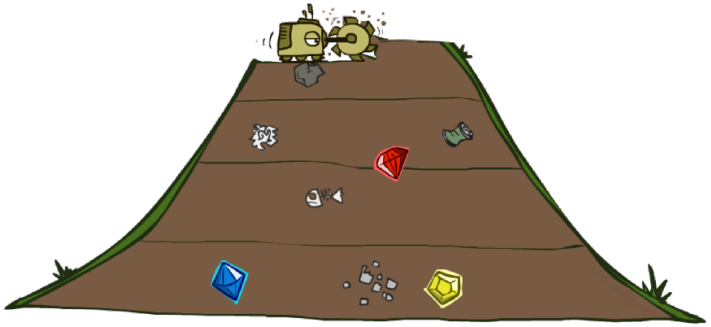


Tree Search

Quiz: DFS versus BFS

When will BFS outperform DFS?





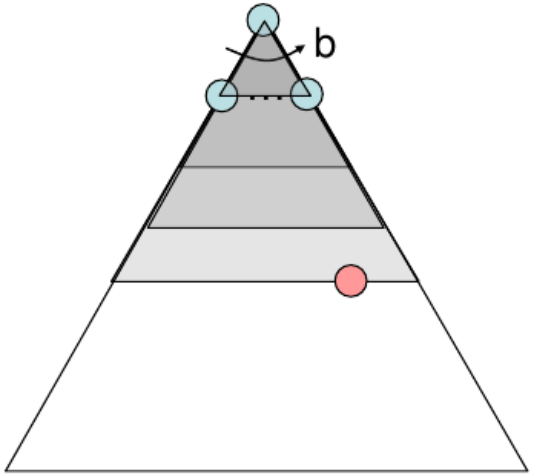
When will DFS outperform BFS?

Tree Search

Iterative Deepening

Idea: get DFS's space advantage with BFS's time / shallow-solution advantages

- Run a DFS with depth limit 1. If no solution ...
- Run a DFS with depth limit 2. If no solution ...
- Run a DFS with depth limit 3. ...

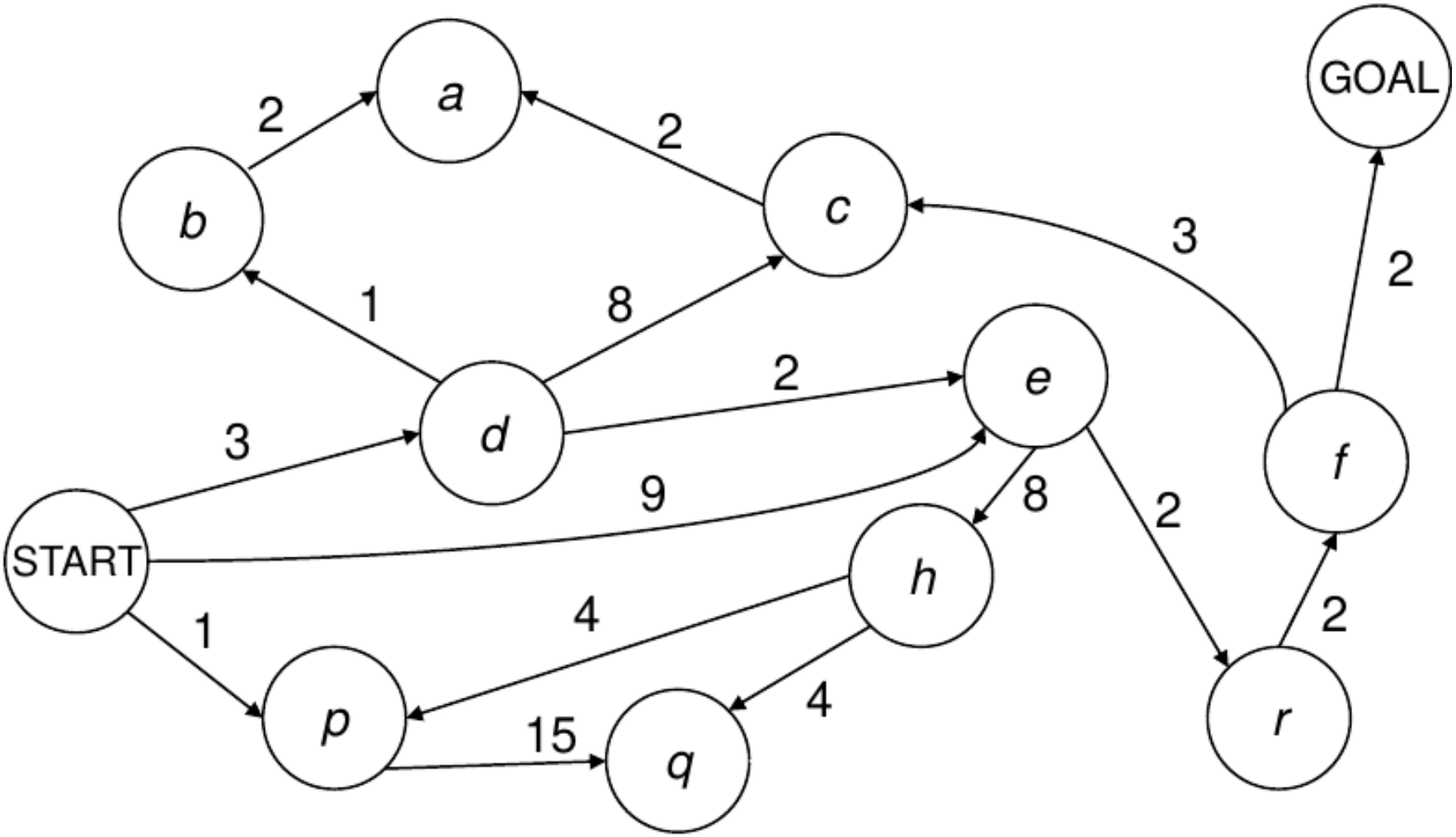


Isn't that wastefully redundant?
Generally most work happens in the lowest level searched, so not so bad!

Content

1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

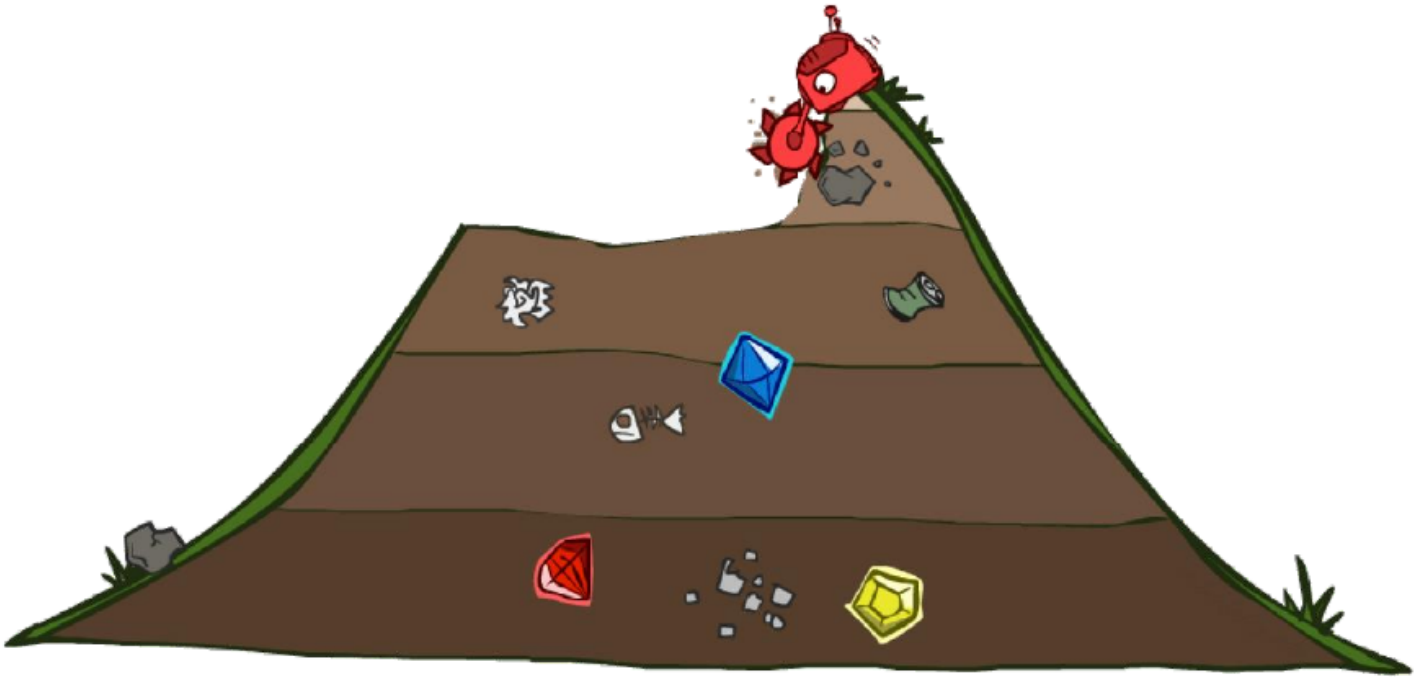
Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path.

Cost-Sensitive Search

Uniform Cost Search (UCS)

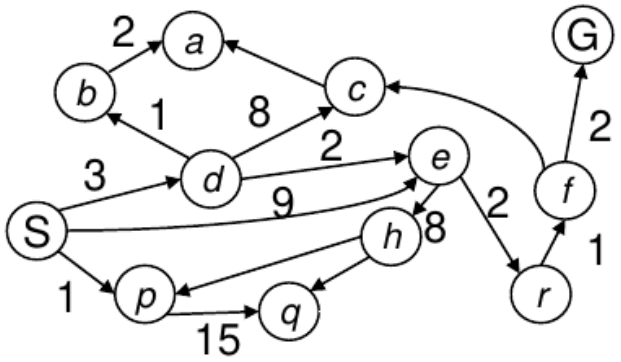


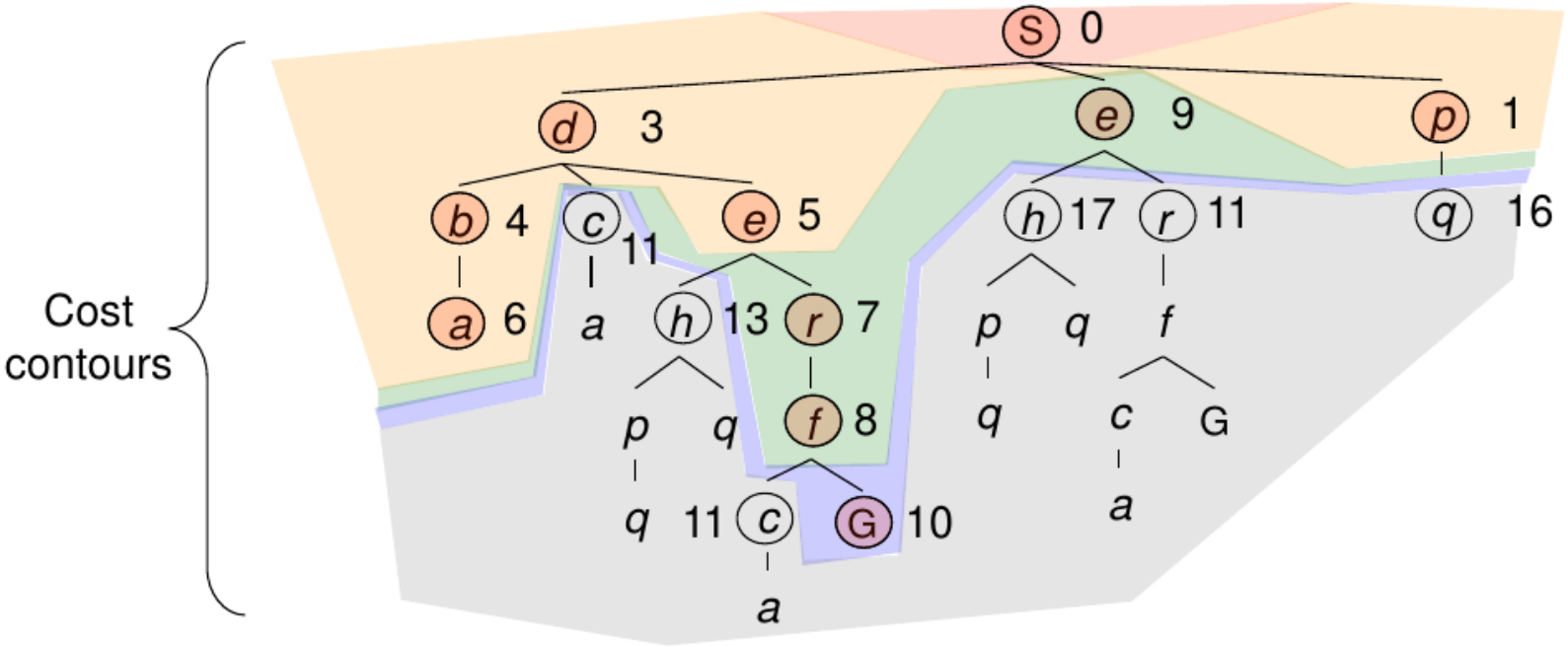
Cost-Sensitive Search

Uniform Cost Search (UCS)

Strategy: expand a **cheapest** node first

Implementation: Frontier is a **priority queue** (priority: cumulative cost)





Cost-Sensitive Search

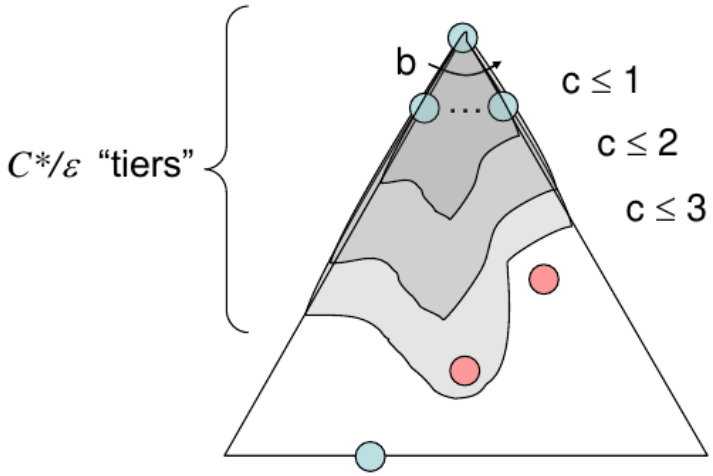
UCS Properties

What nodes UCS expand?

- Processes all nodes with cost less than cheapest solution!
- If that solution costs C^* and arcs cost at least ε, then the "effective depth" is roughly $\frac{C^*}{\varepsilon}$
- Takes time $O\left(b^{C^*/\varepsilon}\right)$ (exponential in effective depth)

How much space does the frontier take?

Has roughly the last tier, so $O\left(b^{C^*/\varepsilon}\right)$
$\Longrightarrow O(bm)$



Complete

Assuming best solution has a finite cost and minimum arc cost is positive, yes!

Optimal

Yes! (Proof next lecture via A*)

Cost-Sensitive Search

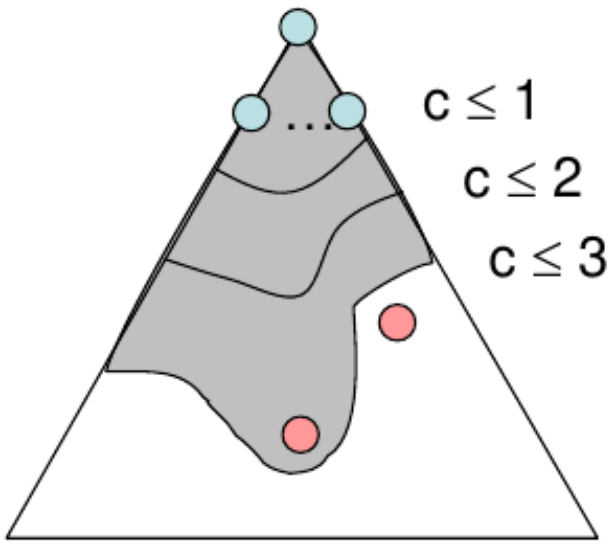
UCS Issue

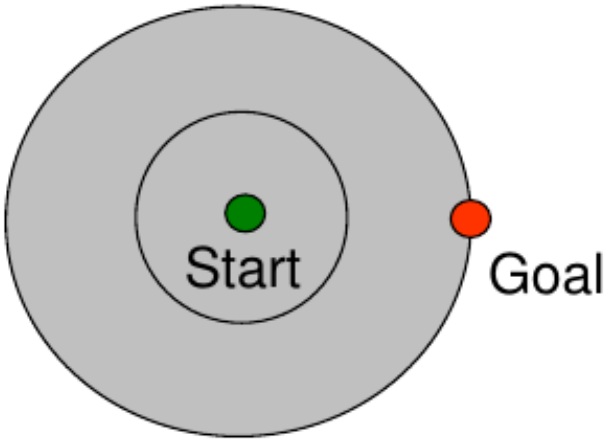
Remember: UCS explores increasing cost contours

The good: UCS is complete and optimal!

The bad:

- Explores options in every "direction"
- No information about goal location





Content

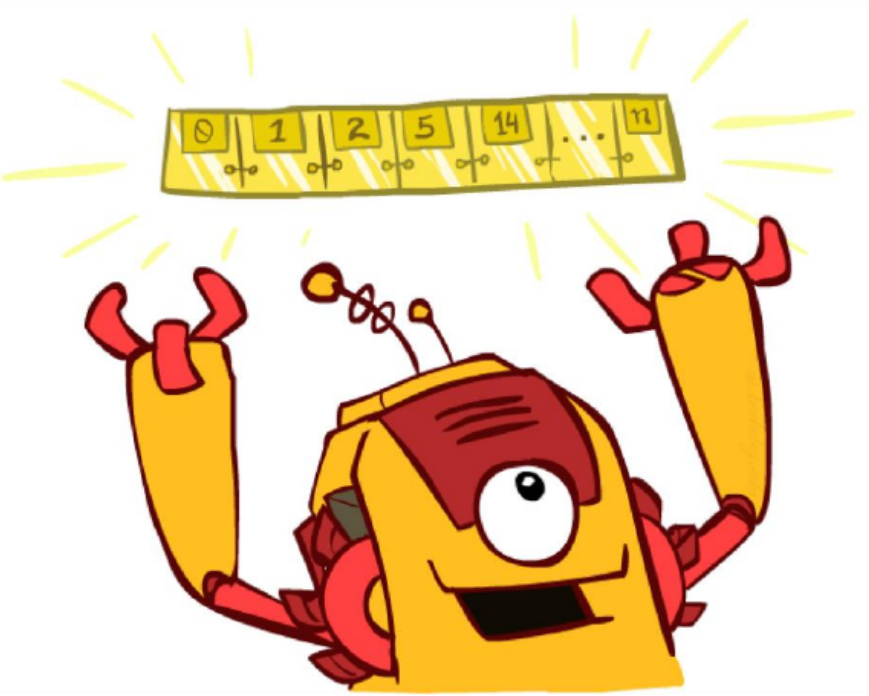
1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

Conclusion

The One Queue

All these search algorithms are the same except for frontier strategies

- Conceptually, all frontiers are priority queues (i.e. collections of nodes with attached priorities)
- Practically, for DFS and BFS, you can avoid the log(n) overhead from an actual priority queue, by using stacks and queues
- Can even code one implementation that takes a variable queuing object



Content

1. Agent
   - Planning
   - Reflex Agent
   - Planning Agent
2. Search Problem
   - Example
   - State Space
   - State Size
   - Data Structure
     - Graph
     - Tree
     - Graph versus Tree
3. Tree Search
   - General Tree Search
   - Depth-First Search
   - Breadth-First Search
   - Iterative Deepening
4. Cost-Sensitive Search
   - Uniform Cost Search
   - Properties
   - Issue
5. Conclusion
6. Exercise

Exercise

Explore the Pacman source code (*JAVA*) and implements the AI of ghosts:

Pinky **Deep-First search**

Clyde **Breadth-First search**

Blinky **Uniform Cost search**